Comprehensive Architectural and Strategic Blueprint for the 2026 GitLab AI Hackathon

Executive Overview and the Automation Imperative

The software development lifecycle is currently undergoing a fundamental and irreversible paradigm shift, transitioning from human-driven, linear processes to asynchronous, AI-orchestrated parallel workflows. This evolution serves as the central premise for the 2026 GitLab AI Hackathon, an online global competition managed by Devpost that challenges developers to engineer intelligent AI agents capable of autonomously navigating complex software development tasks.[1] Scheduled to run from February 9, 2026, with a final submission deadline of March 25, 2026, the event represents a critical testing ground for the generalized deployment of the GitLab Duo Agent Platform.[1]

The industry is currently facing a systemic operational bottleneck identified as the "verification gap".[4] As advanced large language models have commoditized the act of writing boilerplate code, a severe friction point has emerged downstream. AI can generate thousands of lines of code in seconds, but human-driven verification, security auditing, compliance mapping, and deployment processes inherently fail to match this velocity.[4] The hackathon's core theme, encapsulated by the mantra "You Orchestrate, AI Accelerates," mandates the creation of multi-agent systems that do not merely respond in conversational chat interfaces, but react autonomously to environmental triggers and take decisive action across databases, machine learning infrastructures, and productivity domains.[1]

This comprehensive research report provides an exhaustive architectural blueprint and a hyper-accelerated execution strategy designed to conceptualize, engineer, and submit a highly competitive project within a strictly constrained one-week timeframe. The user requirement explicitly demands translating a project scale that typically requires a month of traditional engineering man-hours into a heavily compressed, AI-assisted seven-day sprint. By leveraging advanced natural language programming methodologies—often colloquially referred to as "vibe coding"—alongside a specialized technical stack and a nuanced understanding of the Model Context Protocol, developers can securely orchestrate complex enterprise architectures in a fraction of the traditional time.[5]

Exhaustive Regulatory Framework and Event Parameters

Strict adherence to the hackathon's rules and regulations is the foundational requirement for

any viable submission. Submissions that fail to meet baseline eligibility criteria, circumvent mandatory technological requirements, or violate submission formatting rules are routinely disqualified by the judging panels, regardless of their underlying technological sophistication or conceptual brilliance.[7]

Event Timeline and Participant Eligibility

The competition operates as a public, globally accessible event hosted on the Devpost platform.[1] The official hacking period commences on February 9, 2026, and concludes on March 25, 2026, at 11:30 PM (GMT+5:30).[1] Participants must be at least 18 years of age and can range across the entire spectrum of experience, encompassing university students, seasoned enterprise professionals, and self-taught coders.[8]

Team dynamics are strictly regulated. Teams must consist of a minimum of three and a maximum of five eligible members, although specific Devpost variations and cross-partner hackathons frequently emphasize a four-member limit to maintain balanced collaboration.[8] Pre-formed teams are permitted and actively encouraged, but individuals possess the ability to join organically and be assigned to teams prior to the commencement of the official coding phase. All team structures must be irrevocably finalized before the primary development lifecycle begins.[8]

Prize Infrastructure and Strategic Incentives

The hackathon features a substantial total cash prize pool of $65,000, which is strategically distributed across multiple specialized tracks.[1] Analyzing the financial distribution provides critical insight into the strategic priorities of the event organizers, revealing a distinct push toward multi-partner cloud integrations and sustainable computing practices.[9]

Prize Track	Financial Reward	Technological Requirement	Strategic Implication for Project Architecture
General Prize Pool	Variable based on tier	GitLab Duo Agent Platform	Serves as the baseline requirement for all competitive entries, requiring deep platform integration.

Google Cloud + GitLab	$13,500	GCP Integration	Emphasizes the necessity of advanced cloud computing, scalable infrastructure, and external AI routing.[1]
Anthropic via GitLab	$13,500	Claude/Anthropic APIs	Promotes the integration of advanced Large Language Models for highly complex reasoning and multi-step tasks.[1]
Green Agents	$3,000	Energy Optimization	Highlights the emerging industry mandate for environmentally sustainable AI processing, minimal compute overhead, and token efficiency.[1]

Submission Requirements and Technical Mandates

The overarching project must be engineered entirely from scratch during the designated hackathon period. However, the utilization of open-source libraries, well-documented external APIs, and pre-existing datasets is wholly permitted provided that all external sources are properly credited and licensed.[8] The final submission packet must rigidly adhere to the following components:

1. **Public GitLab Repository:** The entirety of the codebase must be hosted natively on GitLab and configured to be publicly accessible for judging review. A static repository link must be provided in the Devpost submission form.[9]
2. **Demonstration Video:** A video presentation, strictly limited to a maximum of 3 minutes (with some specific regional tracks requiring 2.5 minutes), must be uploaded. This video is critical for showcasing the functional prototype, as judges often rely heavily on the visual proof of autonomous action.[8]
3. **Comprehensive Documentation:** A detailed document must be generated explaining the project's primary objective, identifying its key architectural features, and providing an explicit, itemized breakdown of the specific GitLab tools utilized during development.[11]

A highly critical regulatory mandate surrounds the use of AI development tools. While participants are actively encouraged to utilize assistive tools such as GitHub Copilot, ChatGPT, or the native GitLab Duo for code generation, all AI-generated content must be meticulously documented within the repository.[8] Furthermore, fully AI-generated codebases that entirely lack human oversight, prompt engineering, or human contribution are strictly prohibited.[8] This rule enforces the philosophical core of the event: the human is the strategic orchestrator, while the AI serves as the accelerator. The intellectual merit and architectural integrity must reside definitively with the human developers.[8]

Judging Criteria and Evaluation Metrics

The evaluation framework implemented by Devpost and GitLab relies on a sophisticated matrix combining quantitative performance metrics with qualitative assessments of technological implementation. Projects are rigorously evaluated on their capacity to solve tangible, real-world problems while adhering to stringent operational and security standards.[7]

Evaluation Metric	**Technical Description**	**Strategic Focus and Project Alignment**
Accuracy & F1 Score	Measures prediction correctness, actively balancing the precision and recall of the AI agent's actions.[12]	Ensures agents do not hallucinate workflows, misinterpret codebases, or execute destructive commands autonomously.
Operational Latency	Evaluates the precise time required for an agent to process a trigger and produce a verified response.[12]	Prioritizes highly streamlined, asynchronous communication protocols over monolithic, slow-processing prompts.
Token Efficiency	Assesses the operational efficiency of the model regarding raw token consumption per task.[12]	Directly correlates with the "Green Agents" $3,000 bonus track, requiring strict context-window management.[1]
Conciseness & Coherence	Evaluates the logical consistency, readability, and clarity of the AI outputs and	Validates the effectiveness of the custom system prompts and hierarchical

	resulting code.[12]	instructions.
Technological Implementation	Judges the overarching complexity, sophistication, and utilization of mandatory platform tools.[7]	Requires deep, demonstrable integration with the GitLab Duo Agent Platform and external APIs.[5]

Furthermore, supplementary bonus points—up to a maximum of 5 additional points—are formally awarded to teams that successfully demonstrate the highly effective integration of third-party AI tools (such as the Anthropic API or Google Cloud services) that interact directly with the GitLab platform to enhance development velocity and security.[10]

Architectural Core: The GitLab Duo Agent Platform

To construct a digital architecture capable of dominating this competitive ecosystem, a profound technical mastery of the GitLab Duo Agent Platform and its ancillary technologies is strictly required. The platform acts as the central nervous system for modern software engineering, housing a single application data store that seamlessly orchestrates the entire DevSecOps pipeline from ideation to deployment.[13]

The Duo Agent Platform, currently functioning in a public beta capacity for GitLab Premium and Ultimate customers on both GitLab.com and self-managed environments, fundamentally alters software development by delivering highly extensible, customizable agentic AI capabilities.[10] The platform represents a strategic evolution from earlier, purely conversational models (such as Duo Pro), shifting from isolated one-to-one developer-AI interactions to expansive many-to-many team-agent collaborative networks.[6] The architecture operates across three distinct operational layers:

Foundational Agents

Foundational agents are pre-built, production-ready AI assistants engineered directly by GitLab for common software workflows.[14] These agents arrive with specialized domain expertise and native tools pre-configured, and they are activated by default within the platform, allowing immediate utilization via the GitLab Duo Chat interface.[14] Notable foundational agents include the Planner Agent, designed to parse issues and structure work; the Security Analyst Agent, which automates the triage of vulnerabilities and generates secure patches; and the Data Analyst Agent, which interprets pipeline and repository metrics.[4]

Custom Agents

Custom agents represent the primary vector for hackathon innovation. These are bespoke agents created and configured by the development team to fulfill highly specific operational needs.[14] Developers define the exact behavioral parameters of these agents through natural

language system prompts and deliberately grant them access to specific integrated tools.[16] By equipping a custom agent with tools that interact directly with the GitLab API, the agent gains the autonomous capacity to read files, write new code, formulate commits, and independently open Merge Requests without manual developer intervention.[17] Custom agents are registered within the platform's AI Catalog, a central repository for discovering, creating, and sharing automated workflows across the organization.[6]

External Agents

To extend the baseline capabilities of the platform and directly target the Anthropic and Google Cloud prize tracks, external agents must be utilized.[16] These represent secure connections to third-party AI integrations, allowing the platform to leverage specific external models—such as Anthropic's Claude 3.5 Sonnet or Google's Gemini—for distinct tasks that may require specialized context windows or proprietary reasoning capabilities.[1]

Contextual Engineering: AGENTS.md and System Prompts

The behavioral governance of all agents operating within a repository is dictated by highly specific systemic instructions defined within specialized AGENTS.md configuration files.[18] This configuration architecture operates on a rigorous, cascading tiered hierarchy, allowing instructions to combine and override based on the specific geographical context of the codebase being modified.[18]

The hierarchical structure dictates that agents receive comprehensive context regardless of where they operate within the repository. At the highest level, the User-Level configuration applies globally across all projects associated with a specific developer account, dictating personal coding preferences.[18] Below this sits the Workspace or Project-Level configuration, located in the root directory of the repository. This file defines repository-wide standards, such as mandatory testing frameworks, universal security baselines, and primary architectural patterns.[18]

The most granular control is achieved through Subdirectory-Level configurations. By placing an AGENTS.md file within specific isolated folders (for example, /frontend/AGENTS.md or /backend/AGENTS.md), development teams enforce highly localized conventions.[18] When a GitLab Duo agent attempts to edit files within the /frontend directory, it autonomously reads the root file first, followed by the frontend-specific file. This ensures that an agent modifying frontend React components automatically adheres to entirely different architectural rules than when it is tasked with modifying backend Python microservices.[19]

Best practices for engineering these configuration files dictate that instructions must be kept as concise and specific as possible.[20] Rather than vague mandates, instructions should utilize

strict, actionable language and provide direct syntax examples where necessary, ensuring the agent does not waste token bandwidth interpreting ambiguous commands.[20] Newly added or modified instructions within an AGENTS.md file strictly require the triggering of a new automated flow or the initiation of a fresh chat session to ensure the updated context window is properly loaded.[18]

The Model Context Protocol (MCP): Bridging the Verification Gap

The most critical technological differentiator and primary architectural requirement in the 2026 hackathon landscape is the integration of the Model Context Protocol (MCP).[5] Originally conceptualized and introduced by Anthropic in 2024, MCP is an open-source standard designed to seamlessly and securely connect isolated AI systems with internal enterprise data sources and proprietary tools.[5]

Historically, AI assistants have been inherently limited by their pre-trained data or publicly accessible information. When developers required an AI to contextually understand internal project trackers, bespoke dashboards, secured databases, or proprietary design files, they were forced to manually copy and paste vast amounts of text into the AI's context window—a highly inefficient, token-heavy, and deeply insecure process.[5] MCP fundamentally solves this issue by establishing a secure, bidirectional communication channel. MCP clients (which can be AI applications, autonomous agents, or IDEs) send specific requests for data or actions, and MCP servers provide trusted, fully authorized responses directly from their connected external data sources.[5]

GitLab supports the Model Context Protocol natively through a highly robust, dual-sided architecture:

GitLab as an MCP Client

In the client workflow, the GitLab Duo Agent Platform functions as the MCP client, securely connecting outward to external MCP servers representing tools such as Atlassian Jira, Slack, or Amazon Web Services.[5] This integration allows a GitLab-native agent to autonomously query a Jira backlog, identify high-priority unassigned issues, cross-reference them against Slack communications for context, and independently propose code architecture without ever leaving the GitLab environment.[22] This client-side integration is established via a structured mcp.json file, which maps the external server URLs and connection parameters.[5]

GitLab as an MCP Server

Conversely, in the server workflow, GitLab provides robust MCP server capabilities, enabling external, localized AI tools and applications—such as Claude Desktop, Cursor, or other MCP-compatible IDEs—to securely connect inward to the developer's GitLab instance.[5]

Authentication for this inward connection is rigorously handled through OAuth 2.0 Dynamic Client Registration. When an external AI tool attempts to connect to the GitLab MCP server for the very first time, it automatically registers itself as an OAuth application, formally requests strict authorization scopes to access the GitLab data, and receives a secure access token.[5]

This bidirectional architecture reduces cognitive load and context switching for the developer, vastly accelerates asynchronous workflows, and ensures that the AI can provide deeply meaningful, highly accurate assistance across the entire enterprise toolkit while maintaining strict privacy boundaries.[5] The GitLab MCP server natively supports two primary transport protocols: HTTP transport, which allows for direct connections without the need for additional external dependencies, and stdio transport, which establishes connections through local proxies using Node.js environments.[23]

Multi-Agent Workflows and Orchestration Patterns

The true power of the GitLab Agent Platform lies not in the isolation of individual agents, but in their orchestration into cohesive, multi-agent flows.[24] Flows represent the combination of one or more specialized agents collaborating in a sequential or parallel manner to solve highly complex, multi-step problems autonomously.[15]

Unlike standalone agents, which primarily operate via interactive chat interfaces requiring constant human prompting, flows are designed to work entirely in the background, executing autonomously on the GitLab platform compute infrastructure.[24] Flows are inherently event-driven, engineered to be automatically triggered by specific environmental actions within the repository, such as the assignment of a user, the application of a specific issue label, or the manual request for a code review.[24]

Architectural Aspect	Individual Agents	Autonomous Flows
Interaction Paradigm	Interactive chat requiring human prompting.[24]	Autonomous execution in the background.[24]
Primary Use Case	Seeking guidance, explanations, and performing isolated tasks.[24]	Executing complex, multi-step workflows from end-to-end.[24]
User Involvement	Active, conversational participation.[24]	Setting triggers and reviewing final results.[24]
Execution Timing	Real-time, synchronous responses.[24]	Background, asynchronous processing.[24]

System Complexity	Single-agent, isolated tasks.[24]	Multi-agent, highly orchestrated collaboration.[24]

These autonomous workflows are rigorously defined using custom YAML configurations, enabling development teams to explicitly map out the logic, handoffs, and error-handling protocols between disparate agents.[6] For example, a "Fix CI/CD Pipeline Flow" can be engineered to trigger automatically the moment a build fails. The flow would dispatch an agent to diagnose the logs, dispatch a second agent to write the necessary patch, and dispatch a third agent to push the commit and re-trigger the pipeline, all without a human developer typing a single line of code.[16]

Project Architecture Blueprint: The Rivet System

To maximize the mathematical probability of capturing both the general hackathon prize pool and the highly lucrative specialized bonus tracks (comprising Google Cloud, Anthropic, and the Green Agents initiatives), the proposed project architecture is titled "Rivet."

Rivet is a highly advanced, multi-agent, asynchronous DevSecOps flow explicitly engineered to autonomously transform raw, unstructured Jira business requirements into secure, fully audited, and energy-optimized deployed code. This system directly addresses the hackathon's core mandate to build infrastructure that completely removes friction from the software development lifecycle.[2]

Conceptual Diagram and Orchestration Logic

The system utilizes a complex multi-agent orchestration pattern defined via custom YAML configurations, enabling continuous, autonomous background execution.[6] The architectural data flow is organized in the following sequential framework:

```
-> Jira Issue Tagged "Ready for Engineering" / Webhook Initiated

|

v

+-----------------------------------------------------------------------------+

| THE GITLAB DUO AGENT PLATFORM ORCHESTRATION LAYER |

| |

| |
```

| - Utilizes: MCP Client via HTTP Transport (Atlassian Jira Integration) |

| - Task: Ingests unstructured issue text, reads repository AGENTS.md, and |

| generates a highly structured, step-by-step technical specification. |

| |

| | |

| v |

| |

| - Utilizes: Custom Agent leveraging external Anthropic API (Bonus Track Target) |

| - Task: Ingests technical spec, writes application code, formulates commits, |

| and instantly triggers recursive security scans using Claude 3.5 capabilities.|

| |

| | |

| v |

| |

| - Utilizes: MCP Client (Google Cloud Platform Integration - Bonus Track Target) |

| - Task: Receives merged code, dynamically analyzes GCP spot-instance pricing |

| and carbon-intensity metrics, and executes high-efficiency deployments. |

+----------+

|

v

Granular Component Analysis

The Rivet architecture compartmentalizes distinct cognitive tasks to specialized agents, directly mitigating token bloat and reducing latency.

The first node in the system, the Contextual Planner, acts as the primary data ingestion vector. Leveraging the platform's MCP Client capability, the agent establishes a continuously secure,

bidirectional connection to an external Atlassian Jira server using a customized mcp.json configuration.[5] When a project manager flags an issue as "Ready for Engineering," the Planner agent reads the unstructured text. Crucially, before taking action, it cross-references the overarching repository architecture by reading the project-root AGENTS.md file.[16] It then drafts a comprehensive, multi-step execution plan, effectively eliminating the traditional human task of translating vague business requirements into concrete technical dependencies.

Once the mathematical execution plan is finalized, the YAML workflow immediately transitions control to a Custom Agent powered natively by the Anthropic Claude API, explicitly engineered to secure the $13,500 Anthropic bonus prize.[1] This developer agent is granted highly specific API tools to read files, create distinct branches, and formulate git commits.[17] This agent operates recursively; it writes the necessary codebase, but before utilizing the GitLab API to open a Merge Request, it triggers a secondary, native Security Analyst flow.[16] It analyzes its own newly written code for standard OWASP vulnerabilities, dismisses contextual false positives based on the AGENTS.md guidelines, and auto-generates security patches if flaws are detected.[25]

Upon successful creation of the Merge Request and the passing of all automated CI/CD test suites, the third and final agent assumes control. This deployment agent connects directly via the Model Context Protocol to the Google Cloud Platform, directly targeting the $13,500 GCP integration bonus.[1] Its primary directive is advanced infrastructure optimization. Rather than simply executing a static deployment script, the agent dynamically diagnoses the CI/CD pipeline and actively adjusts runner configurations.[16] It queries GCP metrics to prioritize energy-efficient deployment windows or highly specific, low-carbon compute regions, engineering the deployment specifically to secure the highly competitive $3,000 Green Agents environmental bonus.[1]

By strictly compartmentalizing the tasks (Planner versus Developer versus Deployer), the system avoids the fatal error of feeding the entire repository context into a single, massive prompt window. This architecture drastically reduces raw token consumption, significantly lowers overall API latency, and directly satisfies the most rigorous technical acceptance criteria outlined by the Devpost judges.[12]

Advanced Security Posture and Threat Mitigation

The introduction of fully autonomous agents into the enterprise DevSecOps pipeline fundamentally and exponentially expands a project's potential attack surface. A highly competitive hackathon architecture must account for these novel risks natively, successfully demonstrating to the judging panel that the system is not merely intelligent, but structurally and cryptographically secure against emerging AI-specific threat vectors.[26]

The Lethal Trifecta and Prompt Injection Dynamics

The most severe and pervasive risk to any agentic system is "Prompt Injection." This is a sophisticated attack vector where malicious instructions are deliberately hidden within the unstructured data that an AI is tasked with processing.[26] Instead of following its original, hard-coded system instructions, the AI agent is manipulated into following the hidden commands embedded in the raw data.[26] In the context of the Rivet architecture, a catastrophic prompt injection attack could occur if a bad actor writes a malicious payload directly into a Jira issue ticket description, successfully instructing the Contextual Planner agent to ignore its primary directives and instead execute destructive commands, such as deleting primary repository branches or exfiltrating secure environment variables.[26]

The security architecture mitigates these specific attack vectors through a comprehensive, multi-layered defense-in-depth strategy:

1. **Composite Identity and Least Privilege:** Autonomous agents within the system do not operate with broad or root repository access. Instead, they utilize a strictly controlled composite identity linked directly to a dedicated service account.[26] The internal tools granted to the agent are tightly scoped using least-privilege tokens.[26] If an agent is successfully compromised via prompt injection, it physically lacks the API permissions required to delete the repository or access unauthorized secrets.
2. **Remote Execution Environment Sandboxing:** Any untrusted code executed by the agents during the analysis, generation, or testing phases runs entirely within an isolated, remote Dev Container sandbox. This strictly prevents any malicious code from escaping the temporary environment and infecting the host infrastructure.[26]
3. **Tool Output Sanitization:** Data returned from external sources, such as the Jira MCP Server or the Google Cloud APIs, is aggressively sanitized before being re-ingested into the LLM's primary context window, effectively neutralizing any embedded executable scripts before they can be parsed by the model.[26]
4. **Human-in-the-Loop Governance:** While the underlying YAML flows operate autonomously in the background, highly critical architectural junctures—such as merging a feature branch into the main production branch or executing a live deployment to GCP—strictly require predefined approval rules and cryptographic human sign-off.[26] This directly addresses the industry "verification gap" and ensures stringent regulatory compliance across global standards.[4]

Continuous Compliance and the Pipeline Bill of Materials (PBOM)

Manual auditing of highly complex CI/CD pipelines simply does not scale in an agentic, AI-driven environment.[27] The Rivet system utilizes GitLab's native policy and evidence frameworks to continuously check for enterprise compliance.[27] For large organizations, manual compliance reviews can cost upwards of €100,000 per year in wasted engineering time; automated frameworks are the only sustainable path forward.[27]

Alongside the standard Software Bill of Materials (SBOM), which documents the physical

contents of a compiled artifact, the Rivet system actively generates a Pipeline Bill of Materials (PBOM). The PBOM meticulously documents exactly what digital infrastructure built the artifact: the specific runner images utilized, the reusable actions, the exact pinned versions of the AI agents deployed, and the specific MCP plugins involved in the transaction.[27] This integration ensures that even as autonomous AI dynamically alters the application codebase at high velocity, a permanent, immutable, and fully auditable trail of digital provenance is maintained, satisfying the regulatory mapping requirements for the ISO 27001, NIS2, DORA, and Cyber Resilience Act frameworks.[27]

Accelerated Execution Strategy: Translating Monthly Scope to Weekly Delivery

Compressing a highly complex, multi-agent enterprise architecture—a task that historically demands hundreds of manual engineering hours stretched across a month—into a single, highly compressed week requires extreme operational discipline. This compression is achieved exclusively through the aggressive, continuous utilization of AI-assisted natural language development tools, a process commonly referred to as "vibe coding." By utilizing local IDEs acting as MCP servers connecting to GitLab (such as Cursor or Claude Desktop), developers can seamlessly offload massive amounts of boilerplate generation, syntax debugging, and unit test creation to the AI.[5] In this paradigm, the human orchestrator is elevated from a manual programmer to a high-level architectural director.

Day 1: Environment Scaffolding and Infrastructure Provisioning

The primary objective of the first day is to rapidly establish the secure repository perimeter and initialize the necessary authentication pipelines.

The process begins with the initialization of the public GitLab repository, fulfilling the primary Devpost submission mandate.[11] Immediately, feature flags must be activated at the top-level group settings. By navigating to Group Settings, selecting GitLab Duo, and entering Configuration, the "Allow external MCP tools" feature is activated, enabling all beta and experimental platform capabilities.[16] Following this, Identity and Access Management (IAM) protocols are provisioned. OAuth 2.0 Dynamic Client Registrations are established for the incoming MCP servers, and the requisite API tokens for Google Cloud and the Anthropic console are generated and securely stored.[23]

Crucially, rather than writing server code manually, an external AI-powered IDE connected to the GitLab MCP Server is instructed via natural language to instantly generate the foundational Node.js or Python server architectures required to host the MCP integration layers.[5] The AI autonomously handles all dependency resolutions, parses the package managers, configures the environment variables, and establishes baseline routing structures.

Day 2: Behavioral Engineering and Hierarchical Configuration

The second day focuses entirely on defining the cognitive "minds" of the custom agents. AI coding tools are heavily utilized here to rapidly iterate through various prompt engineering methodologies, dynamically testing semantic variations to maximize the final F1 score accuracy.[12]

The developer instructs the AI to construct the hierarchical AGENTS.md rule system. The root file is generated to define global coding standards, explicitly enforcing asynchronous functions and requiring comprehensive error handling. Subsequent subsystem AGENTS.md files are generated for the isolated frontend and backend microservices.[18] The custom agents are then officially registered within the GitLab AI Catalog. Their system prompts are painstakingly engineered to strictly limit their operational scope; for instance, ensuring the Planner agent cannot physically attempt to write code, and the Security agent cannot attempt to trigger a deployment sequence.[6] Finally, the agents are provisioned with specific GitLab API tools, permanently enabling them to read file structures, create valid commits, and interact directly with the Merge Request API architecture.[17]

Day 3: Model Context Protocol (MCP) Integration and Schema Mapping

This phase is traditionally the most technically complex, historically requiring weeks of API documentation reading and manual JSON integration. Using advanced AI code generation, the tedious translation of external REST or GraphQL endpoints into the standardized MCP format is vastly accelerated.

The developer prompts the AI to build the precise mcp.json file required to establish the secure bridge between the GitLab Duo platform and the external Jira instance.[5] The AI autonomously generates the highly complex schema mapping required to translate Jira's native JSON responses into the strict inputs expected by the GitLab Planner agent. Following this, the http transport mechanisms for the Google Cloud environments are implemented.[23] The developer then utilizes the built-in MCP Dashboard within the VS Code environment to visually verify all connection statuses in real-time, observing the raw server logs to confirm that external tools are being called and executed flawlessly.[22]

Day 4: Multi-Agent YAML Flow Orchestration

With the specialized agents fully defined and all external corporate systems securely connected, the architecture must be physically linked into an autonomous, event-driven chain.

The multi-step workflows are rigorously defined using YAML configuration files (for example, constructing a rivet-flow.yaml file).[6] The event triggers are then configured within the pipeline infrastructure. The developer sets the logic so that the physical creation of a specific issue label

automatically triggers the background, asynchronous execution of the entire agentic flow.[6] To accelerate testing, the developer uses the AI IDE to generate hundreds of mocked webhook events and simulated issue creation scripts, aggressively load-testing the event-driven triggers without the need for manual, slow-paced clicking.

Day 5: Optimization, Green Compute, and Latency Reduction

Because the Devpost evaluation criteria heavily weight operational latency, code conciseness, and token efficiency, the fifth day is strictly dedicated to refining the "Green Agents" parameters.[1]

The context window token consumption of the Planner and Security agents is heavily analyzed. The developer instructs the AI to write data-sanitization middleware scripts that forcefully strip unnecessary metadata and bloated JSON wrappers from the Jira payloads before passing them to the expensive LLM context window. The developer then runs the native "Fix CI/CD Pipeline Flow" [16] and intentionally breaks the software build, actively testing the agent's autonomous recovery protocols and logging the exact time to resolution. Finally, the system verifies that the Google Cloud MCP integration is successfully querying dynamic compute pricing and carbon-intensity APIs before scheduling container deployments, officially satisfying the Green Agents criteria.[1]

Day 6: Documentation Synthesis and Presentation Production

Hackathon success is ultimately dependent on presentation. The judges must thoroughly understand the highly complex architecture within a strictly enforced 3-minute viewing window.[9]

The developer utilizes a Large Language Model to synthesize the entire finalized codebase into the mandatory documentation document. The AI details the primary objective, isolates the key features, and explicitly lists the specific GitLab tools utilized (Duo Platform, MCP Server, YAML flows).[11] Simultaneously, the AI is prompted to script a high-impact, 2.5-minute video presentation. The generated script mathematically allocates 30 seconds to the overarching problem statement, 90 seconds to a live, screen-recorded demonstration of Rivet in action (visually showing a Jira issue turning into a deployed GCP application autonomously), and 30 seconds explicitly detailing the technical stack and bonus track fulfillment. The screen recording captures the GitLab UI, the active MCP dashboard logs, and the GCP console reflecting the automated deployment.[22]

Day 7: Security Audit, Code Freeze, and Final Submission

The final day acts as a strict code freeze. A comprehensive security audit is conducted using automated AI scripts to ensure absolutely no hardcoded API keys, OAuth tokens, or internal passwords were accidentally committed to the public GitLab repository during the rapid

prototyping phase.

The developer ensures that the human verification log—detailing exactly where and how AI coding tools were utilized—is properly formatted and published within the repository's README.md file. This action ensures strict compliance with the Devpost rule forbidding purely non-human code generation.[8] Finally, the Devpost submission form is meticulously completed, embedding the static public repository URL and the finalized YouTube demo video link well prior to the formal March 25, 2026, deadline.[1]

Foundational Implementation Prompts for AI IDE Integration

To execute the "vibe coding" methodology described above, the human orchestrator must feed highly detailed, context-rich system prompts into their AI IDE (such as Cursor or Claude acting as an MCP Server). The following baseline prompts are engineered to instantly bootstrap the architectural scaffolding of the Rivet project.

Prompt 1: Initializing the MCP Server Architecture

> "Acting as an expert DevSecOps architect, generate the complete Node.js boilerplate required to run an MCP Server that interfaces directly with Atlassian Jira and Google Cloud Platform. Utilize the @modelcontextprotocol/sdk package. The server must expose specific tools for query_jira_backlog and fetch_gcp_carbon_metrics. Configure the server to accept connections via stdio transport, and output the required mcp.json file configuration needed to link this server natively to the GitLab Duo Agent Platform. Ensure all code strictly adheres to asynchronous best practices and includes extensive error handling to prevent prompt injection."

Prompt 2: Constructing the Hierarchical AGENTS.md Rulesets

> "Generate a hierarchical set of AGENTS.md files for a GitLab repository. First, generate the root Workspace-level AGENTS.md that strictly enforces Python PEP8 standards, requires comprehensive logging for all actions, and expressly forbids the deletion of any production database rows. Second, generate a Subdirectory-level AGENTS.md specifically for the /frontend directory that enforces React strict mode, mandates the use of functional components over class components, and requires all external API calls to route through a sanitized middleware layer. Keep the instructions highly concise and actionable to minimize LLM token bloat."

Prompt 3: Defining the Multi-Agent YAML Flow Orchestration

> "Create a comprehensive YAML configuration file (rivet-flow.yaml) that defines an autonomous, multi-agent workflow for the GitLab Duo Agent Platform. The flow should trigger automatically when a GitLab issue is labeled with Rivet-Execute. Step 1: Dispatch the custom PlannerAgent to read the issue description. Step 2: Dispatch the DeveloperAgent (utilizing the external Anthropic Claude API) to write the required code and formulate a commit. Step 3: Dispatch the native SecurityAnalystAgent to scan the new commit for OWASP vulnerabilities. Ensure the YAML strictly defines the data handoffs between these agents and includes a fallback protocol if the security scan fails."

Strategic Conclusion

The 2026 GitLab AI Hackathon represents a profound evolutionary milestone in software engineering, officially moving the industry beyond the limitations of conversational AI assistants and into the highly complex realm of autonomous, multi-agent orchestration. The proposed Rivet architecture fully leverages the bleeding-edge capabilities of the GitLab Duo Agent Platform, integrating seamlessly with external enterprise APIs via the Model Context Protocol to secure maximum mathematical viability across both the primary general prize pool and the specialized bonus tracks.

Executing an architectural project of this immense magnitude within a single, compressed week is inherently challenging and breaks traditional project management paradigms. However, by adhering strictly to the structured, AI-accelerated execution strategy outlined within this report, the traditional software development lifecycle is radically compressed. The integration of advanced prompt engineering, pre-configured hierarchical AGENTS.md governance structures, and AI-generated MCP boilerplate codebase allows the human developer to entirely transcend traditional manual coding constraints.

Ultimate success in this global competition relies not merely on writing functional code, but on definitively demonstrating to the judging panel a profound understanding of how to actively govern, cryptographically secure, and seamlessly orchestrate disparate AI systems into a cohesive, latency-optimized, and environmentally efficient DevSecOps workflow. By focusing heavily on stringent regulatory compliance, demonstrating robust security sandboxing against prompt injection vectors, and meticulously documenting the human-AI collaborative process, the resulting submission will stand as a premier, industry-leading example of next-generation software development architecture.

Works cited

1. GitLab AI Hackathon — You Orchestrate. AI Accelerates. - Reddit, accessed on March 12, 2026, https://www.reddit.com/r/hackathon/comments/1r0xzvh/gitlab_ai_hackathon_you_orchestrate_ai_accelerates/
2. GitLab AI Hackathon - Nepvents, accessed on March 12, 2026,

https://www.nepvents.com/events/cmlgk0wjr0003o3mgjzjeg9mi
3. GitLab AI Hackathon Process Walkthrough - YouTube, accessed on March 12, 2026, https://www.youtube.com/watch?v=ilJDxmvapsw
4. From Prompting to Tasking: How the Verification Gap Is Rewriting the Developer's Job, accessed on March 12, 2026, https://medium.com/@random.droid/from-prompting-to-tasking-how-the-verification-gap-is-rewriting-the-developers-job-d78e4145e1f1
5. Model Context Protocol integration - GitLab, accessed on March 12, 2026, https://about.gitlab.com/blog/duo-agent-platform-with-mcp/
6. Get started with GitLab Duo Agent Platform: The complete guide, accessed on March 12, 2026, https://about.gitlab.com/blog/gitlab-duo-agent-platform-complete-getting-started-guide/
7. Understanding hackathon submission and judging criteria: What really counts - Devpost, accessed on March 12, 2026, https://info.devpost.com/blog/understanding-hackathon-submission-and-judging-criteria
8. Rules of Engagement - Intellibus Hackathon, accessed on March 12, 2026, https://www.intellibushackathon.com/rules
9. Dive into innovation: The AI in Action Google Cloud Multi-Partner Hackathon is here, accessed on March 12, 2026, https://info.devpost.com/blog/ai-in-action-google-cloud-multi-partner-hackathon
10. AI in Action Hackathon: Celebrating the GitLab innovations, accessed on March 12, 2026, https://about.gitlab.com/blog/ai-in-action-hackathon-celebrating-the-gitlab-innovations/
11. GitLab Hackathon Challenge - 2025 - Unstop, accessed on March 12, 2026, https://unstop.com/hackathons/i-hack-iit-bombay-1580510
12. AI Evaluation Guidelines | GitLab Docs, accessed on March 12, 2026, https://docs.gitlab.com/development/ai_features/ai_evaluation_guidelines/
13. GitLab Hackathon | PDF | Indemnity - Scribd, accessed on March 12, 2026, https://www.scribd.com/document/977607211/GitLab-Hackathon
14. Agents - GitLab Docs, accessed on March 12, 2026, https://docs.gitlab.com/user/duo_agent_platform/agents/
15. Get started with the GitLab Duo Agent Platform, accessed on March 12, 2026, https://docs.gitlab.com/user/get_started/get_started_agent_platform/
16. GitLab Duo Agent Platform, accessed on March 12, 2026, https://docs.gitlab.com/user/duo_agent_platform/
17. How to Build Custom Agents in GitLab Duo Agent Platform - YouTube, accessed on March 12, 2026, https://www.youtube.com/watch?v=97RXkgeGBPE
18. Customizing GitLab Duo Agent Platform: Chat rules, prompts, and workflows, accessed on March 12, 2026, https://about.gitlab.com/blog/customizing-gitlab-duo-chat-rules-prompts-workflows/

19. AGENTS.md customization files - GitLab Docs, accessed on March 12, 2026, https://docs.gitlab.com/user/duo_agent_platform/customize/agents_md/
20. Customize GitLab Duo Agent Platform, accessed on March 12, 2026, https://docs.gitlab.com/user/duo_agent_platform/customize/
21. Model Context Protocol - GitLab Docs, accessed on March 12, 2026, https://docs.gitlab.com/user/gitlab_duo/model_context_protocol/
22. Extend GitLab Duo Agent Platform: Connect any tool with MCP, accessed on March 12, 2026, https://about.gitlab.com/blog/extend-gitlab-duo-agent-platform-connect-any-tool-with-mcp/
23. GitLab MCP server, accessed on March 12, 2026, https://docs.gitlab.com/user/gitlab_duo/model_context_protocol/mcp_server/
24. Understanding flows: Multi-agent workflows - GitLab, accessed on March 12, 2026, https://about.gitlab.com/blog/understanding-flows-multi-agent-workflows/
25. GitLab Duo Agent Platform, accessed on March 12, 2026, https://about.gitlab.com/gitlab-duo-agent-platform/
26. Security threats in agentic systems - GitLab Docs, accessed on March 12, 2026, https://docs.gitlab.com/user/duo_agent_platform/security_threats/
27. r/gitlab - Reddit, accessed on March 12, 2026, https://www.reddit.com/r/gitlab/